

Java 8 Features

Oracle released a new version of Java as Java 8 in March 18, 2014. It was a revolutionary release of the Java for software development platform. It includes various upgrades to the Java programming, JVM, Tools and libraries.

On April 20, 2009, it was announced that Oracle would acquire Sun for US\$7.4 billion. The deal was completed on January 27, 2010.

What advantages does Java 8 bring?

- Code is more concise and readable
- Code is more reusable
- Code is more testable and maintainable
- Code is now both callable and concurrent
- Users can write parallel code
- Users can write database-like operations
- Applications now perform better
- Code is far more productive
-

With the introduction of Lambda expressions, Stream API, and new Date and Time API, Java 8 has **improved code efficiency and readability**.

Java 8 new Features

- Default methods,
- Static methods in interface
- Functional interfaces,
- Stream API,
- Lambda expressions,
- Method references,
- ForEach() method,
- Optional class,
- Collectors class,
- Nashorn JavaScript Engine,
- Parallel Array Sorting,
- Type and Repating Annotations,
- IO Enhancements,
- Concurrency Enhancements,
- Base64 Encode Decode,

- JDBC Enhancements etc.

Java Functional Interfaces

An Interface that contains exactly one abstract method is known as functional interface. It can have any number of default, static methods but can contain only one abstract method. It can also declare methods of object class.

Functional Interface is also known as Single Abstract Method Interfaces or SAM Interfaces. It is a new feature in Java, which helps to achieve functional programming approach.

A **functional interface** is an interface that contains only one abstract method. They can have only one functionality to exhibit. From Java 8 onwards, lambda expressions can be used to represent the instance of a functional interface. A functional interface can have any number of default methods. **Runnable**, **ActionListener**, and **Comparable** are some of the examples of functional interfaces.

Some Built-in Java Functional Interfaces

Since Java SE 1.8 onwards, there are many interfaces that are converted into functional interfaces. All these interfaces are annotated with `@FunctionalInterface`. These interfaces are as follows –

- **Runnable** → This interface only contains the `run()` method.
- **Comparable** → This interface only contains the `compareTo()` method.
- **ActionListener** → This interface only contains the `actionPerformed()` method.
- **Callable** → This interface only contains the `call()` method.

Functional Interface is also known as Single Abstract Method Interfaces or SAM Interfaces. It is a new feature in Java, which helps to achieve functional programming approach.

Example 1

1. `@FunctionalInterface`
2. **interface** sayable{
3. **void** say(String msg);
4. }
5. **public class** FunctionalInterfaceExample **implements** sayable{
6. **public void** say(String msg){
7. System.out.println(msg);

Prepare by : Vivek Krishna Misra, AP, CSE, DGI, Greater Noida

```

8.     }
9.     public static void main(String[] args) {
10.         FunctionalInterfaceExample fie = new FunctionalInterfaceExample();
11.         fie.say("Hello there");
12.     }
13.}

```

Output:

```

Hello there

```

A functional interface can have methods of object class. See in the following example.

Example 2

```

1.
2. @FunctionalInterface
3. interface sayable{
4.     void say(String msg); // abstract method
5.     // It can contain any number of Object class methods.
6.     int hashCode();
7.     String toString();
8.     boolean equals(Object obj);
9. }
10. public class FunctionalInterfaceExample2 implements sayable{
11.     public void say(String msg){
12.         System.out.println(msg);
13.     }
14.     public static void main(String[] args) {
15.         FunctionalInterfaceExample2 fie = new FunctionalInterfaceExample2();
16.         fie.say("Hello there");
17.     }
18.}

```

Output:

```

Hello there

```

Invalid Functional Interface

A functional interface can extend another interface only when it does not have any abstract method.

```
1. interface sayable{
2.     void say(String msg); // abstract method
3. }
4. @FunctionalInterface
5. interface Doable extends sayable{
6.     // Invalid '@FunctionalInterface' annotation; Doable is not a functional interface
7.     void doIt();
8. }
```

Output:

compile-time error

Example 3

In the following example, a functional interface is extending to a non-functional interface.

```
1. interface Doable{
2.     default void doIt(){
3.         System.out.println("Do it now");
4.     }
5. }
6. @FunctionalInterface
7. interface Sayable extends Doable{
8.     void say(String msg); // abstract method
9. }
10. public class FunctionalInterfaceExample3 implements Sayable{
11.     public void say(String msg){
12.         System.out.println(msg);
13.     }
14.     public static void main(String[] args) {
15.         FunctionalInterfaceExample3 fie = new FunctionalInterfaceExample3();
16.         fie.say("Hello there");
17.         fie.doIt();
18.     }
19. }
```

Output:

```
Hello there  
Do it now
```

Java Lambda Expressions

Lambda is an anonymous function.

- No Name
- No Modifier
- No Return Type

Lambda expression is a new and important feature of Java which was included in Java SE 8. It provides a clear and concise way to represent one method interface using an expression. It is very useful in collection library. It helps to iterate, filter and extract data from collection.

The Lambda expression is used to provide the implementation of an interface which has functional interface. It saves a lot of code. In case of lambda expression, we don't need to define the method again for providing the implementation. Here, we just write the implementation code.

Java lambda expression is treated as a function, so compiler does not create .class file

Why use Lambda Expression

1. To provide the implementation of Functional interface.
2. Less coding.

Java Lambda Expression Syntax

1. (argument-list) -> {body}

Java lambda expression is consisted of three components.

- 1) Argument-list:** It can be empty or non-empty as well.
- 2) Arrow-token:** It is used to link arguments-list and body of expression.
- 3) Body:** It contains expressions and statements for lambda expression.

No Parameter Syntax

```
() -> {
```

```
//Body of no parameter lambda
}
```

One Parameter Syntax

```
(p1) -> {
//Body of single parameter lambda
}
```

Two Parameter Syntax

```
(p1,p2) -> {
//Body of multiple parameter lambda
}
```

For example : method

```
public void int sum(int a,int b)
{
return a+b;
}
```

Can be converted into Lambda as follows

```
(a,b)->a+b;
```

Without Lambda Expression

1. **interface** Drawable{
2. **public void** draw();
3. }
4. **public class** LambdaExpressionExample {
5. **public static void** main(String[] args) {
6. **int** width=10;
- 7.
8. //without lambda, Drawable implementation using anonymous class
9. Drawable d=**new** Drawable(){
10. **public void** draw(){System.out.println("Drawing "+width);}
11. };
12. d.draw();
13. }

14. }

Java Lambda Expression Example

Now, we are going to implement the above example with the help of Java lambda expression.

```
1. @FunctionalInterface //It is optional
2. interface Drawable{
3.     public void draw();
4. }
5.
6. public class LambdaExpressionExample2 {
7.     public static void main(String[] args) {
8.         int width=10;
9.
10.        //with lambda
11.        Drawable d2=()->{
12.            System.out.println("Drawing "+width);
13.        };
14.        d2.draw();
15.    }
16. }
```

A lambda expression can have zero or any number of arguments. Let's see the examples:

Java Lambda Expression Example: No Parameter

```
1. interface Sayable{
2.     public String say();
3. }
4. public class LambdaExpressionExample3{
5.     public static void main(String[] args) {
6.         Sayable s=()->{
7.             return "I have nothing to say.";
8.         };
9.         System.out.println(s.say());
10.    }
11. }
```

Output:

```
I have nothing to say.
```

Java Lambda Expression Example: Single Parameter

```
1. interface Sayable{
2.     public String say(String name);
3. }
4.
5. public class LambdaExpressionExample4{
6.     public static void main(String[] args) {
7.
8.         // Lambda expression with single parameter.
9.         Sayable s1=(name)->{
10.             return "Hello, "+name;
11.         };
12.         System.out.println(s1.say("Sonoo"));
13.
14.         // You can omit function parentheses
15.         Sayable s2= name ->{
16.             return "Hello, "+name;
17.         };
18.         System.out.println(s2.say("Sonoo"));
19.     }
20. }
```

Output:

```
Hello, Sonoo
Hello, Sonoo
```

Java Lambda Expression Example: Multiple Parameters

```
1. interface Addable{
2.     int add(int a,int b);
3. }
4.
5. public class LambdaExpressionExample5{
6.     public static void main(String[] args) {
```

Prepare by : Vivek Krishna Misra, AP, CSE, DGI, Greater Noida

```

7.
8.    // Multiple parameters in lambda expression
9.    Addable ad1=(a,b)->(a+b);
10.   System.out.println(ad1.add(10,20));
11.
12.   // Multiple parameters with data type in lambda expression
13.   Addable ad2=(int a,int b)->(a+b);
14.   System.out.println(ad2.add(100,200));
15. }
16. }

```

Output:

```

30
300

```

Java Lambda Expression Example: with or without return keyword

In Java lambda expression, if there is only one statement, you may or may not use return keyword. You must use return keyword when lambda expression contains multiple statements.

```

1. interface Addable{
2.     int add(int a,int b);
3. }
4.
5. public class LambdaExpressionExample6 {
6.     public static void main(String[] args) {
7.
8.         // Lambda expression without return keyword.
9.         Addable ad1=(a,b)->(a+b);
10.        System.out.println(ad1.add(10,20));
11.
12.        // Lambda expression with return keyword.
13.        Addable ad2=(int a,int b)->{
14.            return (a+b);
15.        };
16.        System.out.println(ad2.add(100,200));
17.    }
18. }

```

Output:

```
30
300
```

Java Lambda Expression Example: Foreach Loop

```
1. import java.util.*;
2. public class LambdaExpressionExample7{
3.     public static void main(String[] args) {
4.
5.         List<String> list=new ArrayList<String>();
6.         list.add("ankit");
7.         list.add("mayank");
8.         list.add("irfan");
9.         list.add("jai");
10.
11.        list.forEach(
12.            (n)->System.out.println(n)
13.        );
14.    }
15.}
```

Output:

```
ankit
mayank
irfan
jai
```

Java Lambda Expression Example: Multiple Statements

```
1. @FunctionalInterface
2. interface Sayable{
3.     String say(String message);
4. }
5.
6. public class LambdaExpressionExample8{
7.     public static void main(String[] args) {
8.
9.         // You can pass multiple statements in lambda expression
           Prepare by : Vivek Krishna Misra, AP, CSE, DGI, Greater Noida
```

```

10.     Sayable person = (message)-> {
11.         String str1 = "I would like to say, ";
12.         String str2 = str1 + message;
13.         return str2;
14.     };
15.     System.out.println(person.say("time is precious.));
16. }
17.}

```

Output:

```
I would like to say, time is precious.
```

Java Lambda Expression Example: Creating Thread

You can use lambda expression to run thread. In the following example, we are implementing run method by using lambda expression.

```

1. public class LambdaExpressionExample9{
2.     public static void main(String[] args) {
3.
4.         //Thread Example without lambda
5.         Runnable r1=new Runnable(){
6.             public void run(){
7.                 System.out.println("Thread1 is running...");
8.             }
9.         };
10.        Thread t1=new Thread(r1);
11.        t1.start();
12.        //Thread Example with lambda
13.        Runnable r2=()->{
14.            System.out.println("Thread2 is running...");
15.        };
16.        Thread t2=new Thread(r2);
17.        t2.start();
18.    }
19.}

```

Output:

```
Thread1 is running...
Thread2 is running...
```

Java lambda expression can be used in the collection framework. It provides efficient and concise way to iterate, filter and fetch data. Following are some lambda and collection examples provided.

Java Lambda Expression Example: Comparator

```
1. import java.util.ArrayList;
2. import java.util.Collections;
3. import java.util.List;
4. class Product{
5.     int id;
6.     String name;
7.     float price;
8.     public Product(int id, String name, float price) {
9.         super();
10.        this.id = id;
11.        this.name = name;
12.        this.price = price;
13.    }
14. }
15. public class LambdaExpressionExample10{
16.     public static void main(String[] args) {
17.         List<Product> list=new ArrayList<Product>();
18.
19.         //Adding Products
20.         list.add(new Product(1,"HP Laptop",25000f));
21.         list.add(new Product(3,"Keyboard",300f));
22.         list.add(new Product(2,"Dell Mouse",150f));
23.
24.         System.out.println("Sorting on the basis of name...");
25.
26.         // implementing lambda expression
27.         Collections.sort(list,(p1,p2)->{
28.             return p1.name.compareTo(p2.name);
29.         });
30.         for(Product p:list){
31.             System.out.println(p.id+" "+p.name+" "+p.price);
32.         }
```

```
33.  
34. }  
35.}
```

Output:

```
Sorting on the basis of name...  
2 Dell Mouse 150.0  
1 HP Laptop 25000.0  
3 Keyboard 300.0
```

Java Lambda Expression Example: Filter Collection Data

```
1. import java.util.ArrayList;  
2. import java.util.List;  
3. import java.util.stream.Stream;  
4. class Product{  
5.     int id;  
6.     String name;  
7.     float price;  
8.     public Product(int id, String name, float price) {  
9.         super();  
10.        this.id = id;  
11.        this.name = name;  
12.        this.price = price;  
13.    }  
14. }  
15. public class LambdaExpressionExample11{  
16.     public static void main(String[] args) {  
17.         List<Product> list=new ArrayList<Product>();  
18.         list.add(new Product(1,"Samsung A5",17000f));  
19.         list.add(new Product(3,"Iphone 6S",65000f));  
20.         list.add(new Product(2,"Sony Xperia",25000f));  
21.         list.add(new Product(4,"Nokia Lumia",15000f));  
22.         list.add(new Product(5,"Redmi4 ",26000f));  
23.         list.add(new Product(6,"Lenevo Vibe",19000f));  
24.  
25.         // using lambda to filter data  
26.         Stream<Product> filtered_data = list.stream().filter(p -> p.price > 20000);  
27.
```

```
28.    // using lambda to iterate through collection
29.    filtered_data.forEach(
30.        product -> System.out.println(product.name+": "+product.price)
31.    );
32. }
33.}
```

Output:

```
Iphone 6S: 65000.0
Sony Xperia: 25000.0
Redmi4 : 26000.0
```

Java Method References

Java provides a new feature called method reference in Java 8. Method reference is used to refer method of functional interface. It is compact and easy form of lambda expression. Each time when you are using lambda expression to just referring a method, you can replace your lambda expression with method reference. In this tutorial, we are explaining method reference concept in detail.

Types of Method References

There are following types of method references in java:

1. Reference to a static method.
2. Reference to an instance method.
3. Reference to a constructor.

1) Reference to a Static Method

You can refer to static method defined in the class. Following is the syntax and example which describe the process of referring static method in Java.

Syntax

1. `ContainingClass::staticMethodName`

Example 1

In the following example, we have defined a functional interface and referring a static method to it's functional method `say()`.

1. **interface** Sayable{

Prepare by : Vivek Krishna Misra, AP, CSE, DGI, Greater Noida

```
2.   void say();
3. }
4. public class MethodReference {
5.   public static void saySomething(){
6.     System.out.println("Hello, this is static method.");
7.   }
8.   public static void main(String[] args) {
9.     // Referring static method
10.    Sayable sayable = MethodReference::saySomething;
11.    // Calling interface method
12.    sayable.say();
13.  }
14. }
```

Output:

```
Hello, this is static method.
```

Example 2

In the following example, we are using predefined functional interface Runnable to refer static method.

```
1. public class MethodReference2 {
2.   public static void ThreadStatus(){
3.     System.out.println("Thread is running...");
4.   }
5.   public static void main(String[] args) {
6.     Thread t2=new Thread(MethodReference2::ThreadStatus);
7.     t2.start();
8.   }
9. }
```

Output:

```
Thread is running...
```

Example 3

You can also use predefined functional interface to refer methods. In the following example, we are using BiFunction interface and using its apply() method.

```

1. import java.util.function.BiFunction;
2. class Arithmetic{
3. public static int add(int a, int b){
4. return a+b;
5. }
6. }
7. public class MethodReference3 {
8. public static void main(String[] args) {
9. BiFunction<Integer, Integer, Integer>adder = Arithmetic::add;
10. int result = adder.apply(10, 20);
11. System.out.println(result);
12. }
13.}

```

Output:

30

Example 4

You can also override static methods by referring methods. In the following example, we have defined and overloaded three add methods.

```

1. import java.util.function.BiFunction;
2. class Arithmetic{
3. public static int add(int a, int b){
4. return a+b;
5. }
6. public static float add(int a, float b){
7. return a+b;
8. }
9. public static float add(float a, float b){
10. return a+b;
11.}
12.}
13.public class MethodReference4 {
14. public static void main(String[] args) {
15. BiFunction<Integer, Integer, Integer>adder1 = Arithmetic::add;
16. BiFunction<Integer, Float, Float>adder2 = Arithmetic::add;
17. BiFunction<Float, Float, Float>adder3 = Arithmetic::add;

```

```

18. int result1 = adder1.apply(10, 20);
19. float result2 = adder2.apply(10, 20.0f);
20. float result3 = adder3.apply(10.0f, 20.0f);
21. System.out.println(result1);
22. System.out.println(result2);
23. System.out.println(result3);
24. }
25. }

```

Output:

```

30
30.0
30.0

```

2) Reference to an Instance Method

like static methods, you can refer instance methods also. In the following example, we are describing the process of referring the instance method.

Syntax

1. containingObject::instanceMethodName

Example 1

In the following example, we are referring non-static methods. You can refer methods by class object and anonymous object.

```

1. interface Sayable{
2.     void say();
3. }
4. public class InstanceMethodReference {
5.     public void saySomething(){
6.         System.out.println("Hello, this is non-static method.");
7.     }
8.     public static void main(String[] args) {
9.         InstanceMethodReference methodReference = new InstanceMethodRef
           erence(); // Creating object
10.        // Referring non-static method using reference
11.        Sayable sayable = methodReference::saySomething;
12.        // Calling interface method
13.        sayable.say();

```

Prepare by : Vivek Krishna Misra, AP, CSE, DGI, Greater Noida

```

14.      // Referring non-static method using anonymous object
15.      Sayable sayable2 = new InstanceMethodReference()::saySomething;
      // You can use anonymous object also
16.      // Calling interface method
17.      sayable2.say();
18.  }
19.}

```

Output:

```

Hello, this is non-static method.
Hello, this is non-static method.

```

Example 2

In the following example, we are referring instance (non-static) method. Runnable interface contains only one abstract method. So, we can use it as functional interface.

```

1. public class InstanceMethodReference2 {
2.     public void printnMsg(){
3.         System.out.println("Hello, this is instance method");
4.     }
5.     public static void main(String[] args) {
6.         Thread t2=new Thread(new InstanceMethodReference2()::printnMsg);
7.         t2.start();
8.     }
9. }

```

Output:

```

Hello, this is instance method

```

Example 3

In the following example, we are using BiFunction interface. It is a predefined interface and contains a functional method apply(). Here, we are referring add method to apply method.

```

1. import java.util.function.BiFunction;
2. class Arithmetic{
3.     public int add(int a, int b){
4.         return a+b;

```

```

5. }
6. }
7. public class InstanceMethodReference3 {
8. public static void main(String[] args) {
9. BiFunction<Integer, Integer, Integer>adder = new Arithmetic().::add;
10. int result = adder.apply(10, 20);
11. System.out.println(result);
12. }
13.}

```

Output:

30

3) Reference to a Constructor

You can refer a constructor by using the new keyword. Here, we are referring constructor with the help of functional interface.

Syntax

```
1. ClassName::new
```

Example

```

1. interface Messageable{
2.     Message getMessage(String msg);
3. }
4. class Message{
5.     Message(String msg){
6.         System.out.print(msg);
7.     }
8. }
9. public class ConstructorReference {
10. public static void main(String[] args) {
11.     Messageable hello = Message::new;
12.     hello.getMessage("Hello");
13. }
14.}

```

Output:

Introduced in Java 8, Stream API is used to process collections of objects. A stream in Java is a sequence of objects that supports various methods which can be pipelined to produce the desired result.

Use of Stream in Java

There uses of Stream in Java are mentioned below:

1. Stream API is a way to express and process collections of objects.
2. Enable us to perform operations like filtering, mapping, reducing and sorting.

How to Create Java Stream?

Java Stream Creation is one of the most basic steps before considering the functionalities of the Java Stream. Below is the syntax given on how to declare Java Stream.

Syntax

```
Stream<T> stream;
```

Here T is either a class, object, or data type depending upon the declaration.

Java Stream Features

The features of Java stream are mentioned below:

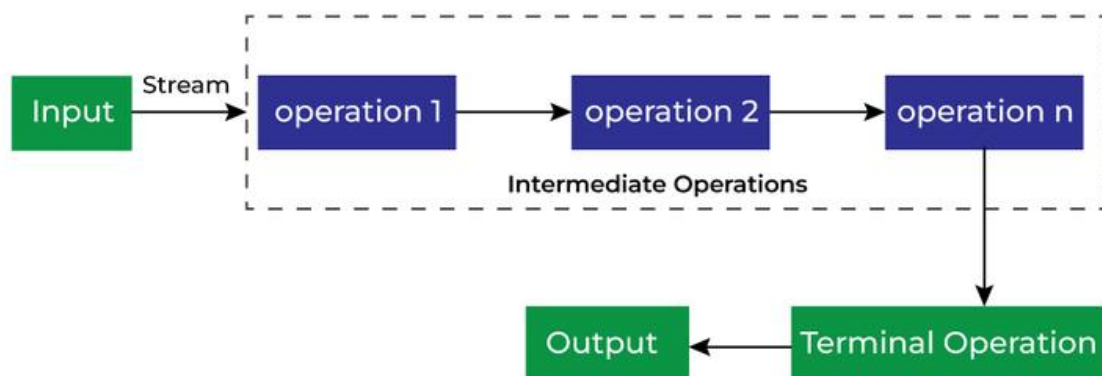
- A stream is not a data structure instead it takes input from the Collections, Arrays or I/O channels.
- Streams don't change the original data structure, they only provide the result as per the pipelined methods.
- Each intermediate operation is lazily executed and returns a stream as a result, hence various intermediate operations can be pipelined. Terminal operations mark the end of the stream and return the result.

Different Operations On Streams

There are two types of Operations in Streams:

1. Intermediate Operations
2. Terminate Operations

Intermediate Operations



Intermediate Operations are the types of operations in which multiple methods are chained in a row.

Characteristics of Intermediate Operations

1. Methods are chained together.
2. Intermediate operations transform a stream into another stream.
3. It enables the concept of filtering where one method filters data and passes it to another method after processing.

Benefit of Java Stream

There are some benefits because of which we use Stream in Java as mentioned below:

- No Storage
- Pipeline of Functions
- Laziness
- Can be infinite
- Can be parallelized
- Can be created from collections, arrays, Files Lines, Methods in Stream, IntStream etc.

Important Intermediate Operations

There are a few Intermediate Operations mentioned below:

1. map()

The map method is used to return a stream consisting of the results of applying the given function to the elements of this stream.

```
List number = Arrays.asList(2,3,4,5);  
List square = number.stream().map(x->x*x).collect(Collectors.toList());
```

2. filter()

The filter method is used to select elements as per the Predicate passed as an argument.

```
List names = Arrays.asList("Reflection","Collection","Stream");  
List result = names.stream().filter(s->s.startsWith("S")).collect(Collectors.toList());
```

3. sorted()

The sorted method is used to sort the stream.

```
List names = Arrays.asList("Reflection","Collection","Stream");  
List result = names.stream().sorted().collect(Collectors.toList());
```

Terminal Operations

Terminal Operations are the type of Operations that return the result. These Operations are not processed further just return a final result value.

Important Terminal Operations

There are a few Terminal Operations mentioned below:

1. collect()

The collect method is used to return the result of the intermediate operations performed on the stream.

```
List number = Arrays.asList(2,3,4,5,3);  
Set square = number.stream().map(x->x*x).collect(Collectors.toSet());
```

2. forEach()

The forEach method is used to iterate through every element of the stream.

```
List number = Arrays.asList(2,3,4,5);  
number.stream().map(x->x*x).forEach(y->System.out.println(y));
```

3. *reduce()*

The reduce method is used to reduce the elements of a stream to a single value. The reduce method takes a BinaryOperator as a parameter.

```
List number = Arrays.asList(2,3,4,5);  
int even = number.stream().filter(x->x%2==0).reduce(0,(ans,i)-> ans+i);
```

Here ans variable is assigned 0 as the initial value and i is added to it.

Java forEach Method

Java provides a new method `forEach()` to iterate the elements. It is defined in `Iterable` and `Stream` interface. It is a default method defined in the `Iterable` interface. Collection classes which extends `Iterable` interface can use `forEach` loop to iterate elements.

This method takes a single parameter which is a functional interface. So, you can pass lambda expression as an argument.

forEach() Signature in Iterable Interface

1. **default void** `forEach(Consumer<super T> action)`
-

Java 8 `forEach()` example 1

1. **import** `java.util.ArrayList;`
2. **import** `java.util.List;`
3. **public class** `ForEachExample` {
4. **public static void** `main(String[] args)` {
5. `List<String> gamesList = new ArrayList<String>();`
6. `gamesList.add("Football");`
7. `gamesList.add("Cricket");`
8. `gamesList.add("Chess");`
9. `gamesList.add("Hockey");`
10. `System.out.println("-----Iterating by passing lambda expression-----`
 `");`
11. `gamesList.forEach(games -> System.out.println(games));`
- 12.
13. }
14. }

Output:

```
-----Iterating by passing lambda expression-----  
Football  
Cricket  
Chess
```

Java 8 forEach() example 2

```

1. import java.util.ArrayList;
2. import java.util.List;
3. public class ForEachExample {
4.     public static void main(String[] args) {
5.         List<String> gamesList = new ArrayList<String>();
6.         gamesList.add("Football");
7.         gamesList.add("Cricket");
8.         gamesList.add("Chess");
9.         gamesList.add("Hocky");
10.        System.out.println("-----Iterating by passing method reference-----
        ");
11.        gamesList.forEach(System.out::println);
12.    }
13.}

```

Output:

```

-----Iterating by passing method reference-----
Football
Cricket
Chess
Hocky

```

Java Stream forEachOrdered() Method

Along with `forEach()` method, Java provides one more method `forEachOrdered()`. It is used to iterate elements in the order specified by the stream.

Singnature:

```

1. void forEachOrdered(Consumer<? super T> action)

```

Java Stream forEachOrdered() Method Example

```

1. import java.util.ArrayList;
2. import java.util.List;
3. public class ForEachOrderedExample {
4.     public static void main(String[] args) {
5.         List<String> gamesList = new ArrayList<String>();
6.         gamesList.add("Football");

```

Prepare by : Vivek Krishna Misra, AP, CSE, DGI, Greater Noida

```

7.     gamesList.add("Cricket");
8.     gamesList.add("Chess");
9.     gamesList.add("Hocky");
10.    System.out.println("-----Iterating by passing lambda expression-----
    ");
11.    gamesList.stream().forEachOrdered(games -
    > System.out.println(games));
12.    System.out.println("-----Iterating by passing method reference-----
    ");
13.    gamesList.stream().forEachOrdered(System.out::println);
14. }
15.
16. }

```

Output:

```

-----Iterating by passing lambda expression-----
Football
Cricket
Chess
Hocky
-----Iterating by passing method reference-----
Football
Cricket
Chess
Hocky

```

Java Default Methods

Java provides a facility to create default methods inside the interface. Methods which are defined inside the interface and tagged with default are known as default methods. These methods are non-abstract methods.

Java Default Method Example

In the following example, Sayable is a functional interface that contains a default and an abstract method. The concept of default method is used to define a method with default implementation. You can override default method also to provide more specific implementation for the method.

Let's see a simple

```

1. interface Sayable{
2.     // Default method
3.     default void say(){

```

```

4.     System.out.println("Hello, this is default method");
5. }
6. // Abstract method
7. void sayMore(String msg);
8. }
9. public class DefaultMethods implements Sayable{
10. public void sayMore(String msg){ // implementing abstract method
11.     System.out.println(msg);
12. }
13. public static void main(String[] args) {
14.     DefaultMethods dm = new DefaultMethods();
15.     dm.say(); // calling default method
16.     dm.sayMore("Work is worship"); // calling abstract method
17.
18. }
19.}

```

Output:

```

Hello, this is default method
Work is worship

```

Static Methods inside Java 8 Interface

You can also define static methods inside the interface. Static methods are used to define utility methods. The following example explain, how to implement static method in interface?

```

1. interface Sayable{
2.     // default method
3.     default void say(){
4.         System.out.println("Hello, this is default method");
5.     }
6.     // Abstract method
7.     void sayMore(String msg);
8.     // static method
9.     static void sayLouder(String msg){
10.         System.out.println(msg);
11.     }
12. }
13. public class DefaultMethods implements Sayable{

```

```

14. public void sayMore(String msg){    // implementing abstract method
15.     System.out.println(msg);
16. }
17. public static void main(String[] args) {
18.     DefaultMethods dm = new DefaultMethods();
19.     dm.say();                // calling default method
20.     dm.sayMore("Work is worship");    // calling abstract method
21.     Sayable.sayLouder("Helloooo..."); // calling static method
22. }
23.}

```

Output:

```

Hello there
Work is worship
Helloooo...

```

Abstract Class vs Java 8 Interface

After having default and static methods inside the interface, we think about the need of abstract class in Java. An interface and an abstract class is almost similar except that you can create constructor in the abstract class whereas you can't do this in interface.

```

1. abstract class AbstractClass{
2.     public AbstractClass() {    // constructor
3.         System.out.println("You can create constructor in abstract class");
4.     }
5.     abstract int add(int a, int b); // abstract method
6.     int sub(int a, int b){    // non-abstract method
7.         return a-b;
8.     }
9.     static int multiply(int a, int b){ // static method
10.        return a*b;
11.    }
12.}
13.public class AbstractTest extends AbstractClass{
14.    public int add(int a, int b){    // implementing abstract method
15.        return a+b;
16.    }
17.    public static void main(String[] args) {
18.        AbstractTest a = new AbstractTest();

```

```

19.    int result1 = a.add(20, 10);    // calling abstract method
20.    int result2 = a.sub(20, 10);    // calling non-abstract method
21.    int result3 = AbstractClass.multiply(20, 10); // calling static method
22.    System.out.println("Addition: "+result1);
23.    System.out.println("Substraction: "+result2);
24.    System.out.println("Multiplication: "+result3);
25. }
26. }

```

Output:

```

You can create constructor in abstract class
Addition: 30
Substraction: 10
Multiplication: 200

```

Static Methods

Static Methods in Interface are those methods, which are defined in the interface with the keyword `static`. Unlike other methods in Interface, these static methods contain the complete definition of the function and since the definition is complete and the method is static, therefore these methods cannot be overridden or changed in the implementation class.

Similar to Default Method in Interface, the static method in an interface can be defined in the interface, but cannot be overridden in Implementation Classes. To use a static method, Interface name should be instantiated with it, as it is a part of the Interface only.

Below programs illustrate static methods in interfaces:

Program 1: To demonstrate use of Static method in Interface.

In this program, a simple static method is defined and declared in an interface which is being called in the `main()` method of the Implementation Class `InterfaceDemo`. Unlike the default method, the static method defines in Interface `hello()`, cannot be overridden in implementing the class.

- Java

```

// Java program to demonstrate
// static method in Interface.

```

```

interface NewInterface {

```

```

    // static method

```

```

    static void hello()

```

```

{
    System.out.println("Hello, New Static Method Here");
}

// Public and abstract method of Interface
void overrideMethod(String str);
}

// Implementation Class
public class InterfaceDemo implements NewInterface {

    public static void main(String[] args)
    {
        InterfaceDemo interfaceDemo = new InterfaceDemo();

        // Calling the static method of interface
        NewInterface.hello();

        // Calling the abstract method of interface
        interfaceDemo.overrideMethod("Hello, Override Method here");
    }

    // Implementing interface method

    @Override
    public void overrideMethod(String str)
    {
        System.out.println(str);
    }
}

```

Output:

Hello, New Static Method Here

Program 2: To demonstrate Scope of Static method.

In this program, the scope of the static method definition is within the interface only. If same name method is implemented in the implementation class then that method becomes a static member of that respective class.

- Java

```
// Java program to demonstrate scope
// of static method in Interface.

interface PrintDemo {

    // Static Method
    static void hello()
    {
        System.out.println("Called from Interface PrintDemo");
    }
}

public class InterfaceDemo implements PrintDemo {

    public static void main(String[] args)
    {

        // Call Interface method as Interface
        // name is preceding with method
        PrintDemo.hello();

        // Call Class static method
        hello();
    }
}
```

```
// Class Static method is defined
static void hello()
{
    System.out.println("Called from Class");
}
}
```

Optional Class

Every Java Programmer is familiar with NullPointerException. It can crash your code. And it is very hard to avoid it without using too many null checks. So, to overcome this, Java 8 has introduced a new class Optional in **java.util package**. It can help in writing a neat code without using too many null checks. By using Optional, we can specify alternate values to return or alternate code to run. This makes the code more readable because the facts which were hidden are now visible to the developer.

Java

// Java program without Optional Class

```
public class OptionalDemo {
    public static void main(String[] args)
    {
        String[] words = new String[10];
        String word = words[5].toLowerCase();
        System.out.print(word);
    }
}
```

Output:

Exception in thread "main" java.lang.NullPointerException

To avoid abnormal termination, we use the Optional class. In the following example, we are using Optional. So, our program can execute without crashing.

The above program using Optional Class

Java

// Java program with Optional Class

```
import java.util.Optional;
```

// Driver Class

```
public class OptionalDemo {
    // Main Method
    public static void main(String[] args)
    {
        String[] words = new String[10];
```

```
Optional<String> checkNull = Optional.ofNullable(words[5]);

if (checkNull.isPresent()) {
    String word = words[5].toLowerCase();
    System.out.print(word);
}
else
    System.out.println("word is null");
}
```

Output

```
word is null
```

Optional is a container object which may or may not contain a non-null value. You must import **java.util package** to use this class. If a value is present, **isPresent()** will return true and **get()** will return the value. Additional methods that depend on the presence or absence of a contained value are provided, such as **orElse()** which returns a default value if the value is not present, and **ifPresent()** which executes a block of code if the value is present. This is a **value-based** class, i.e their instances are :

- Final and immutable (though may contain references to mutable objects).
- Considered equal solely based on equals(), not based on reference equality(==).
- Do not have accessible constructors.

Static Methods: Static methods are the methods in Java that can be called without creating an object of the class. They are referenced by the class name itself or reference to the object of that class.

Syntax :

```
public static void geek(String name)
{
    // code to be executed....
}
```

// Must have static modifier in their declaration.

// Return type can be int, float, String or user-defined data type.

Important Points: Since Static methods belong to the class, they can be called to without creating the object of the class. Below given are some important points regarding Static Methods :

- Static method(s) are associated with the class in which they reside i.e. they can be called even without creating an instance of the class.
- They are designed with the aim to be shared among all objects created from the same class.
- Static methods can not be overridden. But can be overloaded since they are resolved using static binding by the compiler at compile time.

Switch Statement modification in java 8

In Java 8 strings & enum were introduced in case values and switch statements started to evolve

String Switch Case :

```
String day = "Tuesday";
switch (day) {
    case "Monday":
        System.out.println("Week day");
        break;
    case "Tuesday":
        System.out.println("Week day");
        break;
    case "Wednesday":
        System.out.println("Week day");
        break;
    case "Thursday":
        System.out.println("Week day");
        break;
    case "Friday":
        System.out.println("Week day");
        break;
    case "Saturday":
        System.out.println("Weekend");
        break;
    case "Sunday":
        System.out.println("Weekend");
        break;
    default:
        System.out.println("Unknown");
}
```

Enum Switch Case

```
enum DAYS {
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY
}DAYS days = DAYS.SUNDAY;
switch (days) {
    case MONDAY:
        System.out.println("Weekdays");
        break;
    case TUESDAY:
        System.out.println("Weekdays");
        break;
    case WEDNESDAY:
        System.out.println("Weekdays");
        break;
    case THURSDAY:
        System.out.println("Weekdays");
        break;
    case FRIDAY:
        System.out.println("Weekdays");
        break;
    case SATURDAY:
        System.out.println("Weekends");
        break;
}
```

```
case SUNDAY:
    System.out.println("Weekends");
    break;
default:
    System.out.println("Unknown");
}
```

The *yield* Keyword

We may often use a *switch* statement to convert from one value to another. In earlier versions of Java, this required us to either embed the *switch* in a separate function and use the *return* statement from each case or it required us to assign a temporary variable from each case for use later in the function.

Since Java 14, the *yield* keyword within *switch* expressions gives us a better approach.

The *yield* keyword lets us exit a *switch expression* by returning a value that becomes the value of the *switch* expression.

This means we can assign the value of a *switch* expression to a variable.

Lastly, by using *yield* in a *switch* expression, we get an implicit check that we're covering our cases, which makes our code more robust.

Let's look at some examples.

2.1. *yield* with Arrow Operator

To start, let's say we have the following *enum* and *switch* statement:

```
public enum Number {
    ONE, TWO, THREE, FOUR;
}
```

```
String message;
switch (number) {
    case ONE:
        message = "Got a 1";
        break;
    case TWO:
        message = "Got a 2";
        break;
    default:
        message = "More than 2";
}
```

Copy

Let's convert this to a *switch* expression and use the *yield* keyword along with the arrow operator:

```
String message = switch (number) {
```

```

    case ONE -> {
        yield "Got a 1";
    }
    case TWO -> {
        yield "Got a 2";
    }
    default -> {
        yield "More than 2";
    }
};

```

yield sets the value of the *switch* expression depending on the value of *number*.

2.2. *yield* with Colon Delimiter

We can also create a *switch* expression using *yield* with the colon delimiter:

```

String message = switch (number) {
    case ONE:
        yield "Got a 1";
    case TWO:
        yield "Got a 2";
    default:
        yield "More than 2";
};

```

This code behaves the same as in the previous section. But the arrow operator is clearer and also less prone to forgetting *yield* (or *break*) statements.

We should note that we can't mix colon and arrow delimiters within the same *switch* expression.

3. Exhaustiveness

Another nice feature of using the *switch* expression and *yield* is if we are missing case coverage, we'll see a compile error. Let's remove our *default* case from the arrow operator *switch* expression to check:

```

String message = switch (number) {
    case ONE -> {
        yield "Got a 1";
    }
    case TWO -> {
        yield "Got a 2";
    }
};

```

The above code gives us an error on *number*: “the switch expression does not cover all possible input values”.

The above code gives us an error on *number*: “the switch expression does not cover all possible input values”.

We could add the *default* case back in, or we could specifically cover the rest of the possible values of *number*:

```
String message = switch (number) {  
    case ONE -> {  
        yield "Got a 1";  
    }  
    case TWO -> {  
        yield "Got a 2";  
    }  
    case THREE, FOUR -> {  
        yield "More than 2";  
    }  
};
```